Team White Fang

Competitive Programming Notes

Mohammad Siyam
AUTHOR

Numbers Theory

GCD and LCM

```
int gcd(int a, int b) {
    return b == 0 ? a : gcd(b, a % b);
}

int lcm(int a, int b) {
    return (a / gcd(a, b)) * b;
   }
```

Prime Check (√n)

```
bool is_prime(int n) {
    if (n <= 1) return false;
    for (int i = 2; i * i <= n; ++i)
        if (n % i == 0) return false;
    return true;
}
```

Sieve of Eratosthenes (All Primes ≤ N)

```
const int N = 1e6;
bool is_prime[N + 1];

void sieve() {
    fill(is_prime, is_prime + N + 1, true);
    is_prime[0] = is_prime[1] = false;
    for (int i = 2; i * i <= N; ++i)
        if (is_prime[i])
            for (int j = i * i; j <= N; j += i)
                is_prime[j] = false;
}
```

Binary Exponentiation (a^b % m)

```
long long binpow(long long a, long long b, long long m) {
    a %= m;
    long long res = 1;
    while (b > 0) {
        if (b & 1)
            res = res * a % m;
        a = a * a % m;
        b >>= 1;
    }
    return res;
}
```

Modular Inverse using Fermat (when mod is prime)

```
long long mod_inverse(long long a, long long m) {
```

```
    return binpow(a, m - 2, m);
}
```

Euler's Totient Function (ϕ(n))

```
// Returns ϕ(n) = number of integers ≤ n that are coprime with n
// Example: ϕ(9) = 6  (coprime numbers: 1, 2, 4, 5, 7, 8)

int phi(int n) {
    int result = n;
    for (int i = 2; i * i <= n; ++i) {
        if (n % i == 0) {
            while (n % i == 0)
                n /= i;
            result -= result / i;
        }
    }
    if (n > 1)
        result -= result / n;
    return result;
}
```

Prime Factorization (Using SPF)

```
// Naive Prime Factorization
vector<int> prime_factors(int n) {
    vector<int> factors;
    for (int i = 2; i * i <= n; i++) {
        while (n % i == 0) {
            factors.push_back(i);
            n /= i;
        }
    }
    if (n > 1) factors.push_back(n);
    return factors;
}

// Optimized Factorization using SPF (Smallest Prime Factor)
const int N = 1e6;
int spf[N + 1];

void compute_spf() {
    for (int i = 1; i <= N; i++) spf[i] = i;
    for (int i = 2; i * i <= N; i++) {
        if (spf[i] == i) {
            for (int j = i * i; j <= N; j += i)
                if (spf[j] == j) spf[j] = i;
        }
    }
}

vector<int> get_factors(int x) {
    vector<int> res;
    while (x != 1) {
        res.push_back(spf[x]);
        x /= spf[x];
    }
    return res;
}
```

Chinese Remainder Theorem (CRT)

```
// Returns x such that:
// x % a1 = r1
// x % a2 = r2
// and so on, assuming moduli are pairwise coprime

long long chinese_remainder(const vector<int>& a, const vector<int>&
r) {
    long long prod = 1, res = 0;
    int n = a.size();
    for (int i = 0; i < n; ++i) prod *= a[i];
    for (int i = 0; i < n; ++i) {
        long long pp = prod / a[i];
        res += r[i] * modinv(pp, a[i]) * pp;
    }
    return res % prod;
}
```

Precomputing Factorials (Modular)

```
// Precomputes factorials and inverse factorials modulo MOD

// Allows fast computation of nCr(n, r) % MOD in O(1) after
precompute()

// precompute(); // call once before using nCr
// long long ans = nCr(n, r); // compute nCr(n, r) % MOD in O(1)

const int N = 1e6;
const int MOD = 1e9 + 7;
long long fact[N + 1], invfact[N + 1];

void precompute() {
    fact[0] = 1;
    for (int i = 1; i <= N; i++) fact[i] = fact[i - 1] * i % MOD;
    invfact[N] = modinv(fact[N], MOD);
    for (int i = N - 1; i >= 0; i--) invfact[i] = invfact[i + 1] * (i
+ 1) % MOD;
}

long long nCr(int n, int r) {
    if (r > n) return 0;
    return fact[n] * invfact[r] % MOD * invfact[n - r] % MOD;
}
```

Miller-Rabin Primality Test (for large primes)

```
bool is_prime(long long n) {
    if (n < 2) return false;
    for (long long x : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29}) {
        if (n == x) return true;
        if (n % x == 0) return false;
    }
    long long d = n - 1, s = 0;
    while (d % 2 == 0) d /= 2, ++s;
    for (long long a : {2, 3, 5, 7, 11}) {
        long long x = modpow(a, d, n);
        if (x == 1 || x == n - 1) continue;
        bool composite = true;
```

```
        for (int r = 1; r < s; r++) {
            x = x * x % n;
            if (x == n - 1) {
                composite = false;
                break;
            }
        }
        if (composite) return false;
    }
    return true;
}
```

Tricky Input/Output Methods in C++

Input Until EOF

```
// Reads until the end of input (EOF)
int x;
while (cin >> x) { }
```

Read Line by Line (Including Spaces)

```
string line;
while (getline(cin, line)) {}
```

Read Until Sentinel (like `0` or `-1`)

```
int x;
while (cin >> x && x != 0) {}
```

Clear Input Buffer (After `cin` before `getline`)

```
int x;
cin >> x;
cin.ignore(); // clear leftover '\n' from the buffer

string s;
getline(cin, s); // safe to use after ignore
```

Multiple Space-Separated Words in One Line

```
string line;
getline(cin, line);
stringstream ss(line);

string word;
while (ss >> word) {
    // process each word
}
```

Input with Unknown Rows and Columns

```
vector<vector<int>> matrix;
```

```
string line;
while (getline(cin, line)) {
    stringstream ss(line);
    int val;
    vector<int> row;
    while (ss >> val) row.push_back(val);
    matrix.push_back(row);
}
```

Read a Full Integer Array from One Line

```
int n;
cin >> n;
cin.ignore(); // if you're using getline next

string line;
getline(cin, line);
stringstream ss(line);

vector<int> arr;
int x;
while (ss >> x) arr.push_back(x);
```

Output Format with Precision

```
double pi = 3.141592653589;
cout << fixed << setprecision(6) << pi << '\n';
```

Detect End of Line Without Consuming It

```
while (cin.peek() != '\n' && cin >> x) {
    // read integers on the same line
}
cin.ignore(); // skip '\n' after processing the line
```

Must-Know Algorithms & Techniques

Kadane's Algorithm (Max Subarray Sum)

```
int maxSubArray(vector<int>& a) {
    int max_sum = a[0], cur = a[0];
    for (int i = 1; i < a.size(); i++) {
        cur = max(a[i], cur + a[i]);
        max_sum = max(max_sum, cur);
    }
    return max_sum;

}
```

Prefix Sum Array

```
vector<int> prefix(n);
prefix[0] = a[0];
for (int i = 1; i < n; i++) prefix[i] = prefix[i - 1] + a[i];
```

```
// sum of [l, r] = prefix[r] - prefix[l - 1] (if l > 0)
```

Difference Array (Range Update in O(1))

```
diff[l] += x;
diff[r + 1] -= x;
// then apply prefix sum to get final array
```

Prefix XOR

```
vector<int> xorPrefix(n + 1, 0);
for (int i = 0; i < n; i++)
    xorPrefix[i + 1] = xorPrefix[i] ^ arr[i];

// XOR of subarray [l, r] = xorPrefix[r+1] ^ xorPrefix[l]
```

K-th Smallest Element (QuickSelect)

```
nth_element(a.begin(), a.begin() + k, a.end());
```

Moore's Voting Algorithm (Majority Element > n/2)

```
int candidate = -1, count = 0;
for (int num : a) {
    if (count == 0) candidate = num;
    count += (num == candidate) ? 1 : -1;
}
```

Prefix GCD / Suffix GCD

```
for (int i = 1; i < n; i++) prefix[i] = __gcd(prefix[i - 1], a[i]);
```

Sliding Window (Window Sum)

```
int l = 0, sum = 0;
for (int r = 0; r < n; r++) {
    sum += a[r];
    while (sum > k) {
        sum -= a[l];
        l++;
    }
}
```

Binary Search on Answer

```
int lo = 0, hi = 1e9, ans;
while (lo <= hi) {
    int mid = (lo + hi) / 2;
    if (ok(mid)) {
        ans = mid;
        lo = mid + 1;
    } else {
        hi = mid - 1;
    }
}
```

String Matching Algorithms

KMP (Knuth-Morris-Pratt)

```
// Build LPS (Longest Prefix Suffix) array: vector<int> lps =
buildLPS(pattern);
```

```
vector<int> buildLPS(string pattern) {
    int n = pattern.size(), len = 0;
    vector<int> lps(n, 0);
    for (int i = 1; i < n;) {
        if (pattern[i] == pattern[len])
            lps[i++] = ++len;
        else if (len != 0)
            len = lps[len - 1];
        else
            lps[i++] = 0;
    }

    return lps;
}
```

Z-Algorithm

```
// Computes Z-array: z[i] = length of longest substring starting at i
which is also a prefix, Useful for pattern matching in O(n) time
```

```
vector<int> zFunction(string s) {
    int n = s.size();
    vector<int> z(n);
    int l = 0, r = 0;
    for (int i = 1; i < n; i++) {
        if (i <= r) z[i] = min(r - i + 1, z[i - l]);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]])
            z[i]++;
        if (i + z[i] - 1 > r) l = i, r = i + z[i] - 1;
    }
    return z;
}
```

Graph Algorithms

DFS (Depth-First Search)

```
void dfs(int u, vector<vector<int>>& adj, vector<bool>& vis) {
    vis[u] = true;
    for (int v : adj[u]) {
        if (!vis[v]) dfs(v, adj, vis);
    }
```

BFS (Breadth-First Search)

```
void bfs(int start, vector<vector<int>>& adj, vector<bool>& vis) {
    queue<int> q; q.push(start); vis[start] = true;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : adj[u]) {
            if (!vis[v]) q.push(v), vis[v] = true;
        }
    }
}
```

Dijkstra's Algorithm (Weighted Graph, No Negative Weights)

```
vector<int> dijkstra(int src, vector<vector<pair<int,int>>>& adj, int
n) {
    vector<int> dist(n, INT_MAX); dist[src] = 0;
    priority_queue<pair<int,int>, vector<pair<int,int>>, greater<>>
pq;
    pq.push({0, src});
    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d > dist[u]) continue;
        for (auto [v, w] : adj[u]) {
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pq.push({dist[v], v});
            }
        }
    }
    return dist;
}
```

Bellman-Ford (Handles Negative Weights)

```
vector<int> bellman(int src, vector<tuple<int,int,int>>& edges, int n)
{
    vector<int> dist(n, INT_MAX); dist[src] = 0;
    for (int i = 1; i < n; i++) {
        for (auto [u, v, w] : edges) {
            if (dist[u] != INT_MAX && dist[u] + w < dist[v])
                dist[v] = dist[u] + w;
        }
    }
    return dist;
}
```

Floyd-Warshall (All-Pairs Shortest Paths)

```
const int INF = 1e9;
int n; // number of nodes
```

```
vector<vector<int>> dist(n, vector<int>(n, INF));

// Initialization
for (int i = 0; i < n; i++) dist[i][i] = 0;
for (auto [u, v, w] : edges)
    dist[u][v] = min(dist[u][v], w); // or = w if no multiedges

// Floyd-Warshall
for (int k = 0; k < n; k++)
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            if (dist[i][k] < INF && dist[k][j] < INF)
                dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);

// if (dist[src][dest] == INF) cout << "No path\n";
// else cout << "Shortest distance: " << dist[src][dest] << "\n";
```

Prim's Algorithm (MST) [DENSE] O(E log V)

```
// adj[u] = vector of {neighbor, weight} pairs
// Returns total cost of MST
// Usage: int mstCost = prim(n, adj);

int prim(int n, vector<vector<pair<int,int>>>& adj) {
    vector<int> dist(n, INT_MAX); dist[0] = 0;
    vector<bool> vis(n, false);
    priority_queue<pair<int,int>, vector<pair<int,int>>, greater<>>
pq;
    pq.push({0, 0});
    int cost = 0;

    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (vis[u]) continue;
        vis[u] = true;
        cost += d;
        for (auto [v, w] : adj[u])
            if (!vis[v] && w < dist[v]) {
                dist[v] = w;
                pq.push({w, v});
                }
        }
        return cost;
    }
```

Minimum Spanning Tree – Kruskal's O(E log E)

```
// vector<tuple<int,int,int>> edges; // {weight, u, v} for each
edge

// int n = number of vertices

// int mstCost = kruskal(edges, n);

// mstCost now contains the total weight of the MST
```

```
int find(int u) { return par[u] == u ? u : par[u] = find(par[u]); }

int kruskal(vector<tuple<int,int,int>>& edges, int n) {
    sort(edges.begin(), edges.end());
    vector<int> par(n); iota(par.begin(), par.end(), 0);
    int cost = 0;
    for (auto [w, u, v] : edges) {
        if (find(u) != find(v)) {
            par[find(u)] = find(v);
            cost += w;
        }
    }
    return cost;
}
```

Bridge Finding (Tarjan's Algorithm)

```
// A bridge is an edge whose removal increases the number of connected
components

void dfs(int u, int p, int& timer, vector<vector<int>>& adj,
vector<int>& tin, vector<int>& low, vector<bool>& vis) {
    vis[u] = true;
    tin[u] = low[u] = timer++;
    for (int v : adj[u]) {
        if (v == p) continue;
        if (vis[v]) low[u] = min(low[u], tin[v]);
        else {
            dfs(v, u, timer, adj, tin, low, vis);
            low[u] = min(low[u], low[v]);
            if (low[v] > tin[u]) cout << "Bridge: " << u << " - " << v
<< "\n";
        }
    }
}
```

Topological Sort (Kahn's Algorithm / DFS)

```
// Computes a topological ordering of a DAG
// vis[u] marks visited nodes, st stores the sorted order

void topo(int u, vector<vector<int>>& adj, vector<bool>& vis,
stack<int>& st) {
    vis[u] = true;
    for (int v : adj[u])
        if (!vis[v]) topo(v, adj, vis, st);
    st.push(u); }
```

Connected Components

```
// Counts the number of Connected Components in an undirected graph

int countCC(int n, vector<vector<int>>& adj) {

    vector<bool> vis(n, false);
    int cnt = 0;
    for (int i = 0; i < n; i++)
```

```
        if (!vis[i]) dfs(i, adj, vis), cnt++;
    return cnt;
}
```

Bipartite Check (2-Coloring)

```
// Bipartite graph: vertices can be divided into 2 sets/colors
// so that no edge connects vertices of the same set
// Checks if a graph is Bipartite using 2-coloring (DFS)
// color[u] = -1 means unvisited, 0/1 are the two colors
bool isBipartite(int u, vector<vector<int>>& adj, vector<int>& color)
{
    for (int v : adj[u]) {
        if (color[v] == -1) {
            color[v] = 1 - color[u];
            if (!isBipartite(v, adj, color)) return false;
        }
        else if (color[v] == color[u]) return false;
    }
    return true; }
```

0-1 BFS (When weights are 0 or 1)

```
deque<int> dq; dq.push_front(source);
dist[source] = 0;
while (!dq.empty()) {
    int u = dq.front(); dq.pop_front();
    for (auto [v, w] : adj[u]) {
        if (dist[u] + w < dist[v]) {
            dist[v] = dist[u] + w;
            if (w == 1) dq.push_back(v);
            else dq.push_front(v);
        }
    }
```

} Grid BFS / DFS (4/8 directions)

```
int dx[] = {0, 1, 0, -1};
int dy[] = {1, 0, -1, 0};
```

Shortest Path in DAG

```
void topoSort(int u, vector<vector<pair<int,int>>>& adj, vector<bool>&
vis, stack<int>& st) {
    vis[u] = true;
    for (auto [v, w] : adj[u])
        if (!vis[v]) topoSort(v, adj, vis, st);
    st.push(u);
}

vector<int> shortestPathDAG(int n, int src,
vector<vector<pair<int,int>>>& adj) {
    vector<bool> vis(n, false);
```

```
    stack<int> st;
    for (int i = 0; i < n; ++i)
        if (!vis[i]) topoSort(i, adj, vis, st);

    vector<int> dist(n, INT_MAX);
    dist[src] = 0;

    while (!st.empty()) {
        int u = st.top(); st.pop();
        if (dist[u] != INT_MAX) {
            for (auto [v, w] : adj[u])
                if (dist[u] + w < dist[v]) dist[v] = dist[u] + w;
        }
    }
    return dist;
}
```

Articulation Points (Tarjan's)

```
// Finds all vertices whose removal increases number of connected
components

vector<bool> isArticulation(n, false);
vector<bool> vis(n, false);
vector<int> tin(n), low(n);
int timer = 0; u = current, p = parent

void dfs(int u, int p, vector<vector<int>>& adj) {
    vis[u] = true;
    tin[u] = low[u] = timer++;
    int children = 0;
    for (int v : adj[u]) {
        if (v == p) continue;
        if (vis[v])
            low[u] = min(low[u], tin[v]);
        else {
            dfs(v, u, timer, adj, tin, low, vis, isArticulation);
            low[u] = min(low[u], low[v]);
            if (low[v] >= tin[u] && p != -1)
                isArticulation[u] = true;
            children++;
        }
    }
    if (p == -1 && children > 1)
        isArticulation[u] = true;
}
```

Euler Tour (for Subtree Queries, LCA, etc.)

```
int timer = 0;
vector<int> tin, tout;

void dfsEuler(int u, int p, vector<vector<int>>& adj) {
    tin[u] = ++timer;
    for (int v : adj[u]) {
        if (v != p) dfsEuler(v, u, adj);
    }
    tout[u] = ++timer;
}
```

```
bool isAncestor(int u, int v) {
    return tin[u] <= tin[v] && tout[u] >= tout[v];
}

// Function: isAncestor(u, v) -> true if u is ancestor of v
// Run dfsEuler(root, -1, adj) -> fills tin[] and tout[]
```

Heavy-Light Decomposition (HLD Skeleton)

```
const int N = 1e5;
int sz[N], depth[N], par[N], head[N], pos[N], cur_pos = 0;

void dfsSz(int u, int p, vector<vector<int>>& adj) {
    sz[u] = 1; par[u] = p;
    for (int &v : adj[u]) {
        if (v != p) {
            depth[v] = depth[u] + 1;
            dfsSz(v, u, adj);
            sz[u] += sz[v];
        }
    }
}

void dfsHLD(int u, int h, vector<vector<int>>& adj) {
    head[u] = h; pos[u] = cur_pos++;
    int heavy = -1;
    for (int v : adj[u])
       if (v != par[u] && (heavy == -1 || sz[v] > sz[heavy]))heavy = v;
    if (heavy != -1) dfsHLD(heavy, h, adj);
    for (int v : adj[u])
        if (v != par[u] && v != heavy) dfsHLD(v, v, adj);
    }

    // Heavy-Light Decomposition (HLD) - Purpose & Usage:

    // 1. Path Queries (u -> v)
    //    Example: sum/max/min along path from u to v
    //    How: climb u and v via head[] until they meet, query segment tree
    on pos[] ranges

    // 2. Subtree Queries
    //    Example: sum/max/min over all nodes in subtree of u
    //    How: subtree of u corresponds to continuous range [pos[u],
    pos[u]+sz[u]-1] in flattened array

    // 3. Tree Flattening
    //    pos[u] = position of u in flattened array
    //    head[u] = head of the heavy path containing u
    //    depth[u], par[u], sz[u] give tree info for queries

    // Steps to use:
    // 1. Run dfsSz(root, -1, adj) -> computes sz[], depth[], par[]
    // 2. Run dfsHLD(root, root, adj) -> computes head[] and pos[]
    // 3. Build segment tree / BIT on flattened array pos[]
    // 4. For path or subtree query, use pos[] ranges with segment tree/BIT
```

LCA [Binary Lifting]

```
const int LOG = 20, N = 1e5;
int up[N][LOG], depth[N];

void dfsLCA(int u, int p, vector<vector<int>>& adj) {
    up[u][0] = p;
    for (int i = 1; i < LOG; i++)
        up[u][i] = up[up[u][i - 1]][i - 1];
    for (int v : adj[u]) {
        if (v != p) {
            depth[v] = depth[u] + 1;
            dfsLCA(v, u, adj);
        }
    }
}

int lca(int u, int v) {
    if (depth[u] < depth[v]) swap(u, v);
    for (int i = LOG - 1; i >= 0; i--)
        if (depth[u] - (1 << i) >= depth[v])
            u = up[u][i];
    if (u == v) return u;
    for (int i = LOG - 1; i >= 0; i--)
        if (up[u][i] != up[v][i])
            u = up[u][i], v = up[v][i];
    return up[u][0];
}

// Binary Lifting LCA:
// 1. dfsLCA(root, root, adj) -> fills up[][] and depth[]
// 2. lca(u, v) -> returns lowest common ancestor of u and v
// 3. dist(u,v) = depth[u] + depth[v] - 2*depth[lca(u,v)]
```

DSU (Disjoint Set Union)

```
vector<int> par(n), sz(n, 1);

void make(int v){
    par[v] = v;
    sz[v] = 1;
}

int find(int v) {
    if(par[v] == v) return v;
    return par[v] = find(par[v]);
}

void unite(int u, int v) {
    u = find(u); v = find(v);
    if (u == v) return;
    if (sz[u] < sz[v]) swap(u, v);
    par[v] = u;
    sz[u] += sz[v];
}
// DSU (Disjoint Set Union / Union-Find)
// 1. make(v) -> initialize node v
// 2. find(v) -> returns representative of v's set
// 3. unite(u,v) -> merge sets containing u and v
```

```
// Usage: connected components, cycle detection, Kruskal's MST,
dynamic connectivity
```

Cycle Detection (DFS without Coloring)

```
bool dfsCycle(int u, int p, vector<vector<int>>& adj, vector<bool>&
vis) {
    vis[u] = true;
    for (int v : adj[u]) {
        if (!vis[v]) {
            if (dfsCycle(v, u, adj, vis)) return true;
        }
        else if (v != p) return true;
    }
    return false;

}
```

Kosaraju's Algorithm – Strongly Connected Components (SCC)

```
const int N = 1e5 + 5;
vector<int> g[N], rg[N];  // g = original graph, rg = reversed graph
vector<bool> visited;
vector<int> order, component;

void dfs1(int v) {
    visited[v] = true;
    for (int u : g[v])
        if (!visited[u])
            dfs1(u);
    order.push_back(v);
}

void dfs2(int v) {
    visited[v] = true;
    component.push_back(v);
    for (int u : rg[v])
        if (!visited[u])
            dfs2(u);
}

void kosaraju(int n) {
    visited.assign(n + 1, false);
    order.clear();

    // 1st pass on original graph
    for (int i = 1; i <= n; i++)
        if (!visited[i])
            dfs1(i);

    visited.assign(n + 1, false);

    // 2nd pass on reversed graph
    reverse(order.begin(), order.end());
    for (int v : order) {
        if (!visited[v]) {
            component.clear();
            dfs2(v);
            // process this SCC (component)
```

```
            cout << "SCC: ";
            for (int x : component) cout << x << " ";
            cout << "\n";
        }
    }
}
// Kosaraju's Algorithm for Strongly Connected Components (SCC)
// 1. dfs1(v) -> DFS on original graph, fills finishing order
// 2. dfs2(v) -> DFS on reversed graph, fills nodes of one SCC in
'component'
// 3. kosaraju(n) -> computes all SCCs, can process each SCC as needed
// Usage: detect SCCs, graph condensation, cycle detection, 2-SAT,
etc.
```

2-SAT Algorithm (Implication Graph + Kosaraju's SCC)

```
struct TwoSAT {
    int n;
    vector<vector<int>> g, rg;
    vector<int> comp, order;
    vector<bool> used, assignment;

TwoSAT(int vars) : n(vars), g(2 * n), rg(2 * n), comp(2 * n), used(2
* n), assignment(n) {}

    int var(bool is_true, int id) {
        return 2 * id + (is_true ? 0 : 1);
    }

    void add_clause(int a, bool val_a, int b, bool val_b) {
        int u = var(!val_a, a);
        int v = var(val_b, b);
        g[u].push_back(v);
        rg[v].push_back(u);

        u = var(!val_b, b);
        v = var(val_a, a);
        g[u].push_back(v);
        rg[v].push_back(u);
    }

    void dfs1(int v) {
        used[v] = true;
        for (int u : g[v])
            if (!used[u]) dfs1(u);
        order.push_back(v);
    }

    void dfs2(int v, int label) {
        comp[v] = label;
        for (int u : rg[v])
            if (comp[u] == -1) dfs2(u, label);
    }
```

```
    bool solve() {
        fill(used.begin(), used.end(), false);
        order.clear();
        for (int i = 0; i < 2 * n; i++)
            if (!used[i]) dfs1(i);

        fill(comp.begin(), comp.end(), -1);
        reverse(order.begin(), order.end());
        int label = 0;
        for (int v : order)
            if (comp[v] == -1) dfs2(v, label++);

        for (int i = 0; i < n; i++) {
            if (comp[2*i] == comp[2*i+1])
                return false; // x and ¬x in same SCC
            assignment[i] = comp[2*i] > comp[2*i+1]; // higher =
earlier = true
        } return true;
    }
};


// 2-SAT Algorithm using Implication Graph + Kosaraju's SCC

// Purpose:
// Determine if a boolean formula in 2-CNF is satisfiable and, if yes,
find a valid assignment.

// Key Concepts:
// - Each variable x has two nodes: x (true) and ¬x (false) in the
implication graph.
// - Clauses (a ∨ b) are converted into implications: ¬a → b, ¬b → a
// - Run Kosaraju's SCC on the implication graph
// - If x and ¬x are in the same SCC → UNSAT
// - Otherwise, variable assignment = pick value whose SCC comes later
in topological order

// Functions & Usage:
// - TwoSAT(int vars) -> initializes solver for vars variables
// - add_clause(a, val_a, b, val_b) -> adds clause (a=val_a ∨ b=val_b)
// - solve() -> returns true if formula is satisfiable; also fills
assignment[]
// - assignment[i] = true/false for variable i, valid if solve()
returns true

// Example usage:
// TwoSAT solver(n);
// solver.add_clause(0,true,1,false);   // clause: x0 ∨ ¬x1
// if(solver.solve()) { cout << "SAT\n"; cout << solver.assignment[i];
}
```

Tree Algorithms

Subtree Size Calculation

```
vector<int> sz;

int dfsSize(int u, int p, vector<vector<int>>& adj) {
    sz[u] = 1;
    for (int v : adj[u]) {
        if (v != p) sz[u] += dfsSize(v, u, adj);
    }
    return sz[u];
}
```

Depth & Parent Array (LCA Prep)

```
vector<int> depth, parent;

void dfsDepth(int u, int p, vector<vector<int>>& adj) {
    parent[u] = p;
    for (int v : adj[u]) {
        if (v != p) {
            depth[v] = depth[u] + 1;
            dfsDepth(v, u, adj);
        }
    }
}
```

Diameter of Tree (2 DFS Method)

```
int farthest, maxDepth = -1;

void dfsDiameter(int u, int p, int d, vector<vector<int>>& adj) {
    if (d > maxDepth) maxDepth = d, farthest = u;
    for (int v : adj[u]) if (v != p)
        dfsDiameter(v, u, d + 1, adj);
}

// Call dfsDiameter from any node -> get `farthest`
// Call again from `farthest` -> maxDepth = diameter
```

Tree Height

```
int treeHeight(int u, int p, vector<vector<int>>& adj) {
    int h = 0;
    for (int v : adj[u]) {
        if (v != p) h = max(h, treeHeight(v, u, adj));
    }
    return h + 1;
}
```

Centroid of a Tree

```
// A node v is a centroid if all subtrees formed by removing v have a
size <= n/2

int findCentroid(int u, int p, int total, vector<vector<int>>& adj) {
    for (int v : adj[u]) {
        if (v != p && sz[v] > total / 2)
            return findCentroid(v, u, total, adj);
    }
    return u;
}
```

Tree Flattening (DFS Order for Subtree Query in BIT/SegTree)

```
vector<int> in, out, flat;
int timer = 0;

void dfsFlat(int u, int p, vector<vector<int>>& adj) {
    in[u] = timer++;
    flat[in[u]] = u;
    for (int v : adj[u]) {
        if (v != p) dfsFlat(v, u, adj);
    }
    out[u] = timer - 1;
}

pair<int,int> subtreeRange(int u) {
    return {in[u], out[u]}; // left index & right index
}
```

Rerooting DP (Advanced)

```
// Rerooting DP: compute DP for all nodes as root by shifting parent-
child contributions.

// Step 1: dfs1() → compute subtree DP (bottom-up).
// Step 2: dfs2() → reroot by shifting parent↔child contribution.
// res[u] = final answer when u is treated as root.

vector<int> dp, res, sz;

void dfs1(int u, int p, vector<vector<int>>& adj) {
    dp[u] = 1;
    for (int v : adj[u]) {
        if (v != p) {
            dfs1(v, u, adj);
            dp[u] += dp[v];
        }
    }
}

void dfs2(int u, int p, vector<vector<int>>& adj) {
    res[u] = dp[u];
    for (int v : adj[u]) {


        if (v != p) {
            int dpU = dp[u], dpV = dp[v];
            dp[u] -= dp[v]; dp[v] += dp[u];
```

```
            dfs2(v, u, adj);
            dp[u] = dpU; dp[v] = dpV;
        }
    }
}
```

Fenwick Tree (Binary Indexed Tree – BIT)

Point Update + Prefix Sum Query

```
struct BIT {
    int n;
    vector<int> bit;
    BIT(int _n) : n(_n + 1), bit(n, 0) {}

    void update(int i, int delta) {
        for (; i < n; i += i & -i) bit[i] += delta;
    }

    int query(int i) {
        int sum = 0;
        for (; i > 0; i -= i & -i) sum += bit[i];
        return sum;
    }

    int range(int l, int r) { // sum[l...r]
        return query(r) - query(l - 1);
    }
};
```

Segment Tree (Point Update + Range Query[sum,min,max]

1D Segment Tree – Sum Query

```
struct SegmentTree {
    int n;
    vector<int> tree;

    SegmentTree(int _n) : n(_n), tree(4 * n) {}

    void build(vector<int>& a, int v, int tl, int tr) {
        if (tl == tr) {
            tree[v] = a[tl];
        } else {
            int tm = (tl + tr) / 2;
            build(a, v*2, tl, tm);
            build(a, v*2+1, tm+1, tr);
            tree[v] = tree[v*2] + tree[v*2+1];
        }
    }

    int query(int v, int tl, int tr, int l, int r) {
        if (l > r) return 0;
        if (l == tl && r == tr) return tree[v];
```

```
        int tm = (tl + tr) / 2;
        return query(v*2, tl, tm, l, min(r, tm)) +
               query(v*2+1, tm+1, tr, max(l, tm+1), r);
    }

    void update(int v, int tl, int tr, int pos, int new_val) {
        if (tl == tr) {
            tree[v] = new_val;
        } else {
            int tm = (tl + tr) / 2;
            if (pos <= tm)
                update(v*2, tl, tm, pos, new_val);
            else
                update(v*2+1, tm+1, tr, pos, new_val);
            tree[v] = tree[v*2] + tree[v*2+1];
        }
    }
};
```

Lazy Segment Tree (Range Update + Range Query)

```
struct LazySegTree {
    int n;
    vector<int> tree, lazy;

    LazySegTree(int _n) : n(_n), tree(4 * n), lazy(4 * n) {}

    void push(int v, int tl, int tr) {
        if (lazy[v] != 0) {
            tree[v] += (tr - tl + 1) * lazy[v];
            if (tl != tr) {
                lazy[v*2] += lazy[v];
                lazy[v*2+1] += lazy[v];
            }
            lazy[v] = 0;
        }
    }

    void update(int v, int tl, int tr, int l, int r, int add) {
        push(v, tl, tr);
        if (l > r) return;
        if (l == tl && r == tr) {
            lazy[v] += add;
            push(v, tl, tr);
        } else {
            int tm = (tl + tr) / 2;
            update(v*2, tl, tm, l, min(r, tm), add);
            update(v*2+1, tm+1, tr, max(l, tm+1), r, add);
            tree[v] = tree[v*2] + tree[v*2+1];
        }
    }

    int query(int v, int tl, int tr, int l, int r) {
        push(v, tl, tr);
        if (l > r) return 0;
        if (l <= tl && tr <= r) return tree[v];
        int tm = (tl + tr) / 2;
        return query(v*2, tl, tm, l, min(r, tm)) +
```

```
                query(v*2+1, tm+1, tr, max(l, tm+1), r);
    }
};
```

Sparse Table (Range Min/Max in O(1)/ RMQ)

```
const int LOG = 20;
vector<vector<int>> st;
vector<int> lg;

void buildST(vector<int>& a) {
    int n = a.size();
    st.assign(n, vector<int>(LOG));
    lg.resize(n + 1);
    for (int i = 2; i <= n; i++) lg[i] = lg[i / 2] + 1;
    for (int i = 0; i < n; i++) st[i][0] = a[i];
    for (int j = 1; j < LOG; j++)
        for (int i = 0; i + (1 << j) <= n; i++)
            st[i][j] = min(st[i][j-1], st[i + (1 << (j-1))][j-1]);
}

int queryST(int l, int r) {
    int j = lg[r - l + 1];
    return min(st[l][j], st[r - (1 << j) + 1][j]);
}
// buildST(a) → preprocess
// queryST(l,r) → min value in [l,r]
// static array only (no updates)
```

Trie (String Prefix Tree)

```
struct TrieNode {
    TrieNode* child[26];
    bool isEnd;
    TrieNode() {
        fill(child, child+26, nullptr);
        isEnd = false;
    }
};

void insert(TrieNode* root, string s) {
    for (char c : s) {
        int idx = c - 'a';
        if (!root->child[idx]) root->child[idx] = new TrieNode();
        root = root->child[idx];
    }
    root->isEnd = true;
}

// Trie: insert(root, s) → add string s
// search(root, s) → check full string s exists
// startsWith(root, prefix) → check prefix exists
// Efficient for lowercase a-z, no deletions handled
```

Dynamic Programming (DP)

0/1 Knapsack

```
int knapsack(int W, vector<int>& wt, vector<int>& val, int n) {
    vector<vector<int>> dp(n+1, vector<int>(W+1, 0));
    for (int i = 1; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            if (wt[i-1] <= w)
                dp[i][w] = max(dp[i-1][w], val[i-1] + dp[i-1][w -
wt[i-1]]);
            else
                dp[i][w] = dp[i-1][w];
        }
    }

    return dp[n][W];
}
```

Unbounded Knapsack

```
int unboundedKnapsack(int W, vector<int>& wt, vector<int>& val, int n)
{
    vector<int> dp(W+1, 0);
    for (int w = 0; w <= W; w++) {
        for (int i = 0; i < n; i++) {
            if (wt[i] <= w) dp[w] = max(dp[w], dp[w - wt[i]] +
val[i]);
        }
    }
    return dp[W];
}

// unboundedKnapsack(W, wt, val, n) → max value with total weight ≤ W
// Each item can be used unlimited times
```

Subset Sum / Target Sum

```
bool subset_sum(vector<int>& a, int target) {
    int n = a.size();
    vector<vector<bool>> dp(n+1, vector<bool>(target+1, false));
    dp[0][0] = true;
    for (int i = 1; i <= n; i++) {
        for (int sum = 0; sum <= target; sum++) {
            dp[i][sum] = dp[i-1][sum];
            if (sum >= a[i-1])
                dp[i][sum] |= dp[i-1][sum - a[i-1]];
        }
    }
    return dp[n][target];
}
```

Longest Increasing Subsequence (LIS)

```
int lis(vector<int>& a) {
    vector<int> dp;
```

```
    for (int x : a) {
        auto it = lower_bound(dp.begin(), dp.end(), x);
        if (it == dp.end()) dp.push_back(x);
        else *it = x;
    }
    return dp.size();       }
```

Longest Common Subsequence (LCS)

```
int lcs(string a, string b) {
    int n = a.size(), m = b.size();
    vector<vector<int>> dp(n+1, vector<int>(m+1));
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            dp[i][j] = (a[i-1] == b[j-1]) ? 1 + dp[i-1][j-1] :
max(dp[i-1][j], dp[i][j-1]);
    return dp[n][m];
}
```

Edit Distance

```
int edit_distance(string a, string b) {
    int n = a.size(), m = b.size();
    vector<vector<int>> dp(n+1, vector<int>(m+1));
    for (int i = 0; i <= n; i++)
        for (int j = 0; j <= m; j++) {
            if (i == 0) dp[i][j] = j;
            else if (j == 0) dp[i][j] = i;
            else if (a[i-1] == b[j-1]) dp[i][j] = dp[i-1][j-1];
            else dp[i][j] = 1 + min({dp[i-1][j], dp[i][j-1], dp[i-
1][j-1]});
        }
    return dp[n][m];
}
// Edit Distance (Levenshtein distance)
// edit_distance(a, b) → min number of operations to convert string a
to b
// Operations allowed: insert, delete, replace
// dp[i][j] = edit distance between first i chars of a and first j
chars of b
```

DP on Trees

```
vector<int> g[N];
int dp[N];

void dfs(int u, int p) {
    dp[u] = 0;
    for (int v : g[u]) {
        if (v == p) continue;
        dfs(v, u);
        dp[u] += max(0, dp[v]);
    }
    dp[u] += weight[u]; // Add node value
}
```

Digit DP

```
int dp[20][2][2];

int count(int pos, int tight, int leading_zero, string& num) {
    if (pos == num.size()) return 1;
    if (dp[pos][tight][leading_zero] != -1) return
dp[pos][tight][leading_zero];

    int limit = tight ? (num[pos] - '0') : 9, res = 0;
    for (int d = 0; d <= limit; d++) {
        int ntight = tight & (d == limit);
        int nlead = leading_zero & (d == 0);
        res += count(pos + 1, ntight, nlead, num);
    }
    return dp[pos][tight][leading_zero] = res;
}

// count(pos, tight, leading_zero, num) → count numbers ≤ num with
constraints
// pos = current digit index
// tight = is prefix equal to num so far
// leading_zero = if number has leading zeros
```

Bitmask DP

```
int dp[1 << 20];

int solve(int mask, int n) {
    if (mask == (1 << n) - 1) return 0;
    if (dp[mask] != -1) return dp[mask];
    int ans = INT_MAX;
    for (int i = 0; i < n; i++) {
        if (!(mask & (1 << i))) {
            ans = min(ans, cost[i] + solve(mask | (1 << i), n));
        }
    }
    return dp[mask] = ans;
}
// solve(mask, n) → compute answer for current subset represented by
mask
dp[mask] = answer for subset mask, try adding unset elements
// Iterate over unset bits to try adding next element
```

Divide and Conquer Optimization

```
void compute(int i, int l, int r, int optl, int optr) {
    if (l > r) return;
    int mid = (l + r) / 2;
    pair<long long, int> best = {LLONG_MAX, -1};
    for (int k = optl; k <= min(mid, optr); k++) {
        long long cost = dp[i-1][k] + C[k][mid];
        if (cost < best.first)
            best = {cost, k};
    }
```

```
    dp[i][mid] = best.first;
    compute(i, l, mid-1, optl, best.second);
    compute(i, mid+1, r, best.second, optr);
}
```

Knuth's Optimization

```
vector<vector<long long>> dp(n, vector<long long>(n));
vector<vector<int>> opt(n, vector<int>(n));

for (int l = 1; l < n; l++) {
    for (int i = 0, j = l; j < n; i++, j++) {
        dp[i][j] = LLONG_MAX;
        for (int k = opt[i][j - 1]; k <= opt[i + 1][j]; k++) {
            long long cost = dp[i][k] + dp[k][j] + C[i][j];
            if (cost < dp[i][j]) {
                dp[i][j] = cost;
                opt[i][j] = k;
            }
        }
    }
}
```

SOS DP (Sum Over Subsets)

```
const int N = 1 << 20; // maximum 20 bits
int f[N];  // f[mask] = value for subset mask

// Precompute sum over all subsets of mask
// After this, f[mask] = sum of values of all subsets of mask
void sosDP(int n) {
    for (int i = 0; i < n; i++) {
        for (int mask = 0; mask < (1 << n); mask++) {
            if (mask & (1 << i)) f[mask] += f[mask ^ (1 << i)];
        }
    }
}

//Call sosDP(n) (n = number of bits)
//f[mask] = sum/count over all subsets of mask, O(n*2^n)
```

MCM (Matrix Chain Multiplication)

```
// arr = dimensions of matrices
// arr[i-1] x arr[i] = dimension of matrix i
int mcm(vector<int>& arr) {
    int n = arr.size();
    vector<vector<int>> dp(n, vector<int>(n, 0));

    for (int len = 2; len < n; len++) {  // chain length
        for (int i = 1; i + len - 1 < n; i++) {
            int j = i + len - 1;
            dp[i][j] = INT_MAX;
            for (int k = i; k < j; k++) {
```

```
                dp[i][j] =
min(dp[i][j], dp[i][k] + dp[k+1][j] + arr[i1]*arr[k]*arr[j]);
            }
        }
    }
    return dp[1][n-1];
}
```

Extras

```
    // PRIORITY_QUEUE
    priority_queue<int> pq; // max-heap
    pq.push(5); pq.push(1); pq.push(10);
    cout << "Max PQ Top: " << pq.top() << "\n";


    priority_queue<int, vector<int>, greater<int>> minpq; // min-heap
    minpq.push(5); minpq.push(1); minpq.push(10);
    cout << "Min PQ Top: " << minpq.top() << "\n";

    // BITSET
    bitset<8> b("10101010");
    b.flip(); cout << "Flipped: " << b << "\n";
    cout << "1's count: " << b.count() << "\n";


// Decimal to Binary - returns binary string
string decimalToBinary(int n) {
    if (n == 0) return "0";
    string res = "";
    while (n > 0) {
        res += (n % 2) ? '1' : '0';
        n /= 2;
    }
    reverse(res.begin(), res.end());
    return res;
}
```